TaskFlow implementation - code test basis

SECJ3343 Project 1 - Group TaskFlow. Committed source baseline main@5133a6c.

https://github.com/aland99/TaskFlow

This is a faithful text snapshot of the named repository files used to derive acceptance requirements. Line numbers refer to the committed source. Wrapped continuation lines retain indentation. Source behavior is distinguished from independently documented requirements in each member's test basis.

src/App.css

```
/* Layout and component styles. Colours and shared controls live in index.css. */

.app {
  display: flex;
  min-height: 100vh;
}

/* ---------- Sidebar ---------- */

.sidebar {
  position: sticky;
  top: 0;
  z-index: 30;
  display: flex;
  flex-direction: column;
  flex-shrink: 0;
  width: 248px;
  height: 100vh;
  background: var(--surface);
  border-right: 1px solid var(--border);
  transition: width 0.2s;
}

.sidebar-header {
  display: flex;
  align-items: center;
  gap: 10px;
  min-height: 72px;
  padding: 0 20px;
}

.logo {
  display: inline-flex;
  align-items: center;
  justify-content: center;
  flex-shrink: 0;
  width: 36px;
  height: 36px;
  border-radius: 10px;
  background: var(--primary);
  color: #fff;
}

.brand-name {
  font-size: 20px;
  font-weight: 700;
  letter-spacing: -0.01em;
}

.sidebar-close {
  display: none;
  margin-left: auto;
}

.sidebar-nav {
  display: flex;
  flex-direction: column;
  gap: 4px;
  flex: 1;
  padding: 8px 12px;
}

.nav-item {
  display: flex;
  align-items: center;
  gap: 12px;
  min-height: 44px;
  padding: 0 12px;
  border: 0;
  border-radius: 8px;
  background: transparent;
```

```
  color: var(--text-muted);
  font-size: 15px;
  font-weight: 500;
  text-align: left;
  cursor: pointer;
}
.nav-item:hover {
  background: var(--neutral-tint);
  color: var(--text);
}
.nav-item.active {
  background: var(--primary-tint);
  color: var(--primary-ink);
  font-weight: 600;
}
.nav-item svg {
  flex-shrink: 0;
}

.sidebar-footer {
  display: flex;
  flex-direction: column;
  gap: 8px;
  padding: 12px;
  border-top: 1px solid var(--border);
}

.profile {
  display: flex;
  align-items: center;
  gap: 10px;
  padding: 4px;
  min-width: 0;
}

.avatar {
  display: inline-flex;
  align-items: center;
  justify-content: center;
  flex-shrink: 0;
  width: 36px;
  height: 36px;
  border-radius: 50%;
  background: var(--primary-tint);
  color: var(--primary-ink);
}

.profile-name {
  font-size: 14px;
  font-weight: 600;
  white-space: nowrap;
}
.profile-note {
  font-size: 12px;
  color: var(--text-muted);
  white-space: nowrap;
}

.collapse-button {
  align-self: flex-end;
}

/* Collapsed sidebar (desktop only): icons only */
@media (min-width: 769px) {
  .sidebar.collapsed {
    width: 72px;
  }
  .sidebar.collapsed .brand-name,
  .sidebar.collapsed .nav-label,
  .sidebar.collapsed .profile-text {
    display: none;
  }
  .sidebar.collapsed .sidebar-header {
    justify-content: center;
```

```
    padding: 0;
  }
  .sidebar.collapsed .nav-item {
    justify-content: center;
    padding: 0;
  }
  .sidebar.collapsed .profile {
    justify-content: center;
  }
  .sidebar.collapsed .collapse-button {
    align-self: center;
  }
}

.drawer-overlay {
  display: none;
}

/* ---------- Main area and top bar ---------- */

.main {
  flex: 1;
  min-width: 0;
}

.topbar {
  position: sticky;
  top: 0;
  z-index: 20;
  display: flex;
  align-items: center;
  gap: 16px;
  padding: 14px 32px;
  background: var(--surface);
  border-bottom: 1px solid var(--border);
}

.menu-button {
  display: none;
}

.topbar-title {
  flex: 1;
  min-width: 0;
}
.topbar-title h1 {
  font-size: 22px;
  font-weight: 700;
  line-height: 1.25;
}
.current-date {
  font-size: 13px;
  color: var(--text-muted);
}

.topbar-search,
.search-box {
  position: relative;
  display: flex;
  align-items: center;
}
.topbar-search {
  width: 280px;
}
.topbar-search svg,
.search-box svg {
  position: absolute;
  left: 12px;
  color: var(--text-muted);
  pointer-events: none;
}
.topbar-search input,
.search-box input {
  padding-left: 38px;
```

```
}

.content {
  max-width: 1120px;
  margin: 0 auto;
  padding: 24px 32px 48px;
}

/* ---------- Storage warning banner ---------- */

.banner {
  display: flex;
  align-items: center;
  gap: 12px;
  margin-bottom: 20px;
  padding: 12px 16px;
  border: 1px solid #fcd34d;
  border-radius: var(--radius);
  background: var(--warning-tint);
  color: var(--warning-ink);
}
.banner p {
  flex: 1;
  min-width: 0;
  font-size: 14px;
  font-weight: 500;
}
.banner svg {
  flex-shrink: 0;
}

/* ---------- Summary cards ---------- */

.summary {
  display: grid;
  grid-template-columns: repeat(4, 1fr);
  gap: 16px;
  margin-bottom: 24px;
}

.summary-card {
  display: flex;
  align-items: center;
  gap: 14px;
  min-width: 0;
  padding: 18px 20px;
  background: var(--surface);
  border: 1px solid var(--border);
  border-radius: var(--radius);
  box-shadow: var(--shadow-sm);
}

.summary-icon {
  display: inline-flex;
  align-items: center;
  justify-content: center;
  flex-shrink: 0;
  width: 44px;
  height: 44px;
  border-radius: 12px;
}
.tone-primary {
  background: var(--primary-tint);
  color: var(--primary);
}
.tone-neutral {
  background: var(--neutral-tint);
  color: var(--neutral-ink);
}
.tone-warning {
  background: var(--warning-tint);
  color: var(--warning);
}
.tone-success {
```

```
294   background: var(--success-tint);
295   color: var(--success);
296 }
297
298 .summary-label {
299   font-size: 14px;
300   color: var(--text-muted);
301 }
302 .summary-count {
303   font-size: 28px;
304   font-weight: 700;
305   line-height: 1.2;
306 }
307
308 /* ---------- Toolbar ---------- */
309
310 .toolbar {
311   display: grid;
312   grid-template-columns: minmax(200px, 2fr) repeat(3, minmax(130px, 1fr)) auto;
313   gap: 12px;
314   align-items: end;
315   margin-bottom: 20px;
316   padding: 16px;
317   background: var(--surface);
318   border: 1px solid var(--border);
319   border-radius: var(--radius);
320   box-shadow: var(--shadow-sm);
321 }
322
323 /* ---------- Task list ---------- */
324
325 .result-count {
326   margin-bottom: 10px;
327   font-size: 13px;
328   color: var(--text-muted);
329 }
330
331 .task-list {
332   display: flex;
333   flex-direction: column;
334   gap: 12px;
335   margin: 0;
336   padding: 0;
337   list-style: none;
338 }
339
340 .task-card {
341   display: flex;
342   align-items: flex-start;
343   justify-content: space-between;
344   gap: 16px;
345   padding: 16px 20px;
346   background: var(--surface);
347   border: 1px solid var(--border);
348   border-radius: var(--radius);
349   box-shadow: var(--shadow-sm);
350 }
351 .task-card.overdue {
352   border-left: 4px solid var(--error);
353 }
354
355 .task-main {
356   flex: 1;
357   min-width: 0;
358 }
359
360 .task-head {
361   display: flex;
362   flex-wrap: wrap;
363   align-items: center;
364   gap: 6px 8px;
365 }
366
367 .task-title {
```

```
  margin-right: 4px;
  font-size: 17px;
  font-weight: 600;
  overflow-wrap: anywhere;
}
.task-card.completed .task-title {
  color: var(--text-muted);
}

.task-description {
  margin-top: 6px;
  color: var(--text-muted);
  font-size: 14px;
  white-space: pre-wrap;
  overflow-wrap: anywhere;
}

.task-meta {
  display: flex;
  flex-wrap: wrap;
  gap: 4px 16px;
  margin-top: 10px;
  font-size: 13px;
  color: var(--text-muted);
}
.overdue-text {
  color: var(--error);
  font-weight: 600;
}

.task-actions {
  display: flex;
  flex-wrap: wrap;
  align-items: center;
  justify-content: flex-end;
  gap: 8px;
  flex-shrink: 0;
}
.status-select {
  width: 140px;
  font-size: 14px;
}

/* ---------- Empty states ---------- */

.empty-state {
  display: flex;
  flex-direction: column;
  align-items: center;
  gap: 8px;
  padding: 48px 24px;
  text-align: center;
  background: var(--surface);
  border: 1px dashed var(--border-strong);
  border-radius: var(--radius);
  color: var(--text-muted);
}
.empty-state h3 {
  font-size: 18px;
  color: var(--text);
  overflow-wrap: anywhere;
}
.empty-state p {
  margin-bottom: 8px;
  font-size: 14px;
}

/* ---------- Modals ---------- */

.modal-backdrop {
  position: fixed;
  inset: 0;
  z-index: 50;
  display: flex;
```

```
  align-items: center;
  justify-content: center;
  padding: 16px;
  overflow-y: auto;
  background: rgba(17, 24, 39, 0.5);
}

.modal {
  width: 100%;
  max-width: 520px;
  max-height: calc(100vh - 32px);
  display: flex;
  flex-direction: column;
  background: var(--surface);
  border-radius: 16px;
  box-shadow: var(--shadow-lg);
}
.modal form {
  display: flex;
  flex-direction: column;
  min-height: 0;
}

.modal-header {
  display: flex;
  align-items: center;
  justify-content: space-between;
  padding: 16px 12px 16px 24px;
  border-bottom: 1px solid var(--border);
}
.modal-header h2 {
  font-size: 18px;
}

.modal-body {
  display: flex;
  flex-direction: column;
  gap: 16px;
  padding: 20px 24px;
  overflow-y: auto;
}
.confirm-text {
  overflow-wrap: anywhere;
}

.field-row {
  display: grid;
  grid-template-columns: 1fr 1fr;
  gap: 16px;
}

.modal-footer {
  display: flex;
  justify-content: flex-end;
  gap: 12px;
  padding: 16px 24px;
  border-top: 1px solid var(--border);
}

/* ---------- Toast ---------- */

.toast {
  position: fixed;
  right: 24px;
  bottom: 24px;
  z-index: 60;
  display: flex;
  align-items: center;
  gap: 10px;
  max-width: calc(100vw - 32px);
  padding: 8px 8px 8px 16px;
  border-radius: 10px;
  background: #111827;
  color: #fff;
```

```
  box-shadow: var(--shadow-lg);
  font-size: 14px;
  font-weight: 500;
}
.toast-success > svg {
  color: #4ade80;
}
.toast-error > svg {
  color: #f87171;
}
.toast-message {
  min-width: 0;
  overflow-wrap: anywhere;
}
.toast-close {
  color: #d1d5db;
}
.toast-close:hover {
  background: rgba(255, 255, 255, 0.12);
  color: #fff;
}

/* ---------- Tablet ---------- */

@media (max-width: 1024px) {
  .summary {
    grid-template-columns: repeat(2, 1fr);
  }
  .toolbar {
    grid-template-columns: repeat(3, 1fr);
  }
  .toolbar-search {
    grid-column: 1 / -1;
  }
  .toolbar > .button {
    grid-column: 1 / -1;
  }
  .topbar-search {
    width: 220px;
  }
}

/* Narrow main area: put the task actions under the task details */
@media (max-width: 960px) {
  .task-card {
    flex-direction: column;
    align-items: stretch;
  }
  .task-actions {
    justify-content: flex-start;
  }
}

/* ---------- Mobile: sidebar becomes a drawer ---------- */

@media (max-width: 768px) {
  .sidebar {
    position: fixed;
    top: 0;
    left: 0;
    width: 272px;
    visibility: hidden;
    transform: translateX(-100%);
    transition: transform 0.2s, visibility 0.2s;
    box-shadow: var(--shadow-lg);
  }
  .sidebar.open {
    visibility: visible;
    transform: none;
  }
  .sidebar-close {
    display: inline-flex;
  }
  .collapse-button {
```

```
    display: none;
  }
  .drawer-overlay {
    display: block;
    position: fixed;
    inset: 0;
    z-index: 25;
    background: rgba(17, 24, 39, 0.5);
  }

  .menu-button {
    display: inline-flex;
    margin-left: -8px;
  }
  .topbar {
    padding: 10px 16px;
    gap: 8px;
  }
  .topbar-title h1 {
    font-size: 18px;
  }
  .topbar-search {
    display: none; /* the toolbar search below is always visible on small screens */
  }

  .content {
    padding: 16px 16px 40px;
  }

  .task-card {
    padding: 16px;
  }
  .task-actions {
    justify-content: stretch;
  }
  .status-select {
    flex: 1 1 100%;
    width: auto;
  }
  .task-actions .button {
    flex: 1;
  }

  .toast {
    right: 16px;
    left: 16px;
    bottom: 16px;
  }
  .banner {
    flex-wrap: wrap;
  }
}

@media (max-width: 480px) {
  .summary {
    grid-template-columns: 1fr;
  }
  .toolbar {
    grid-template-columns: 1fr;
  }
  .field-row {
    grid-template-columns: 1fr;
  }
  .modal-footer .button {
    flex: 1;
  }
}
```

src/App.jsx

```jsx
import { useCallback, useRef, useState } from 'react'
import './App.css'
import Sidebar from './components/Sidebar'
import TopBar from './components/TopBar'
import SummaryCards from './components/SummaryCards'
import TaskToolbar from './components/TaskToolbar'
import TaskList from './components/TaskList'
import TaskFormModal from './components/TaskFormModal'
import ConfirmDeleteModal from './components/ConfirmDeleteModal'
import StorageBanner from './components/StorageBanner'
import Toast from './components/Toast'
import { loadTasks, resetStorage, saveTasks } from './utils/storage'
import { buildTask, filterAndSortTasks, normalizeValues } from './utils/tasks'
import { todayString } from './utils/dates'
import { STATUSES } from './utils/validation'

export default function App() {
  // Load once at startup. loadTasks never throws, even if localStorage holds garbage.
  const [initial] = useState(loadTasks)
  const [tasks, setTasks] = useState(initial.tasks)
  const [warning, setWarning] = useState(initial.warning)

  // Search is shared by the top bar and the toolbar.
  const [search, setSearch] = useState('')
  const [statusFilter, setStatusFilter] = useState('All')
  const [priorityFilter, setPriorityFilter] = useState('All')
  const [sort, setSort] = useState('newest')

  const [view, setView] = useState('Dashboard') // 'Dashboard' or 'All Tasks'
  const [collapsed, setCollapsed] = useState(false)
  const [drawerOpen, setDrawerOpen] = useState(false)
  const [modal, setModal] = useState(null) // null | { type: 'create' } | { type: 'edit' | 'delete', id }
  const [toast, setToast] = useState(null)
  const toastCount = useRef(0)

  const showToast = (message, type = 'success') => setToast({ message, type, key: ++toastCount.current })
  const dismissToast = useCallback(() => setToast(null), [])
  const closeModal = () => setModal(null)

  // Every change goes through here: update the screen, save immediately, then show a toast.
  function commit(nextTasks, successMessage) {
    setTasks(nextTasks)
    if (saveTasks(nextTasks)) showToast(successMessage)
    else showToast('Could not save tasks to browser storage.', 'error')
  }

  function createTask(values) {
    commit([buildTask(values), ...tasks], 'Task created successfully.')
    closeModal()
  }

  function updateTask(id, values) {
    const changes = { ...normalizeValues(values), updatedAt: new Date().toISOString() }
    commit(tasks.map((task) => (task.id === id ? { ...task, ...changes } : task)), 'Task updated successfully.')
    closeModal()
  }

  function changeStatus(id, status) {
    if (!STATUSES.includes(status)) return
    const updatedAt = new Date().toISOString()
    commit(tasks.map((task) => (task.id === id ? { ...task, status, updatedAt } : task)), 'Task status updated.')
  }

  function deleteTask(id) {
    commit(tasks.filter((task) => task.id !== id), 'Task deleted successfully.')
    closeModal()
  }
```

```
68
69  function clearFilters() {
70    setSearch('')
71    setStatusFilter('All')
72    setPriorityFilter('All')
73    setSort('newest')
74  }
75
76  // Dashboard and All Tasks show every status; the other three items set the status filter.
77  function navigate(label) {
78    if (STATUSES.includes(label)) {
79      setStatusFilter(label)
80    } else {
81      setView(label)
82      setStatusFilter('All')
83    }
84    setDrawerOpen(false)
85  }
86
87  function handleResetStorage() {
88    resetStorage()
89    setTasks([])
90    setWarning(null)
91  }
92
93  const activeNav = statusFilter === 'All' ? view : statusFilter
94  const visibleTasks = filterAndSortTasks(tasks, { search, status: statusFilter, priority:
    priorityFilter, sort })
95  const editingTask = modal?.type === 'edit' ? tasks.find((task) => task.id === modal.id) : null
96  const deletingTask = modal?.type === 'delete' ? tasks.find((task) => task.id === modal.id) : null
97
98  return (
99    <div className="app">
100     <Sidebar
101       active={activeNav}
102       onNavigate={navigate}
103       collapsed={collapsed}
104       onToggleCollapse={() => setCollapsed(!collapsed)}
105       open={drawerOpen}
106       onClose={() => setDrawerOpen(false)}
107     />
108     {drawerOpen && <div className="drawer-overlay" onClick={() => setDrawerOpen(false)} />}
109
110     <div className="main">
111       <TopBar
112         title={activeNav}
113         search={search}
114         onSearchChange={setSearch}
115         onNewTask={() => setModal({ type: 'create' })}
116         onOpenMenu={() => setDrawerOpen(true)}
117       />
118
119       <main className="content">
120         {warning && (
121           <StorageBanner warning={warning} onReset={handleResetStorage} onDismiss={() =>
    setWarning(null)} />
122         )}
123         <SummaryCards tasks={tasks} />
124         <TaskToolbar
125           search={search}
126           onSearchChange={setSearch}
127           status={statusFilter}
128           onStatusChange={setStatusFilter}
129           priority={priorityFilter}
130           onPriorityChange={setPriorityFilter}
131           sort={sort}
132           onSortChange={setSort}
133           onClear={clearFilters}
134         />
135         <TaskList
136           totalCount={tasks.length}
137           tasks={visibleTasks}
138           search={search}
139           today={todayString()}
```

```
            onEdit={(id) => setModal({ type: 'edit', id })}
            onDelete={(id) => setModal({ type: 'delete', id })}
            onStatusChange={changeStatus}
            onCreate={() => setModal({ type: 'create' })}
            onClearSearch={() => setSearch('')}
            onClearFilters={clearFilters}
          />
        </main>
      </div>

      {modal?.type === 'create' && <TaskFormModal onSave={createTask} onClose={closeModal} />}
      {editingTask && (
        <TaskFormModal
          key={editingTask.id}
          task={editingTask}
          onSave={(values) => updateTask(editingTask.id, values)}
          onClose={closeModal}
        />
      )}
      {deletingTask && (
        <ConfirmDeleteModal task={deletingTask} onConfirm={() => deleteTask(deletingTask.id)}
  onClose={closeModal} />
      )}

      {toast && <Toast key={toast.key} message={toast.message} type={toast.type} onClose={dismissToast}
  />}
    </div>
  )
}
```

src/components/ConfirmDeleteModal.jsx

```jsx
import Modal from './Modal'

export default function ConfirmDeleteModal({ task, onConfirm, onClose }) {
  return (
    <Modal title="Delete Task" onClose={onClose}>
      <div className="modal-body">
        <p className="confirm-text">
          Are you sure you want to delete &amp;quot;{task.title}&amp;quot;? This cannot be undone.
        </p>
      </div>
      <div className="modal-footer">
        <button type="button" className="button button-secondary" onClick={onClose} data-autofocus>
          Cancel
        </button>
        <button type="button" className="button button-danger" onClick={onConfirm}>
          Confirm Delete
        </button>
      </div>
    </Modal>
  )
}
```

src/components/EmptyState.jsx

```jsx
export default function EmptyState({ icon: Icon, title, text, actionLabel, onAction }) {
  return (
    <div className="empty-state">
      <Icon size={40} aria-hidden="true" />
      <h3>{title}</h3>
      <p>{text}</p>
      <button type="button" className="button button-primary" onClick={onAction}>
        {actionLabel}
      </button>
    </div>
  )
}
```

src/components/Modal.jsx

```jsx
import { useEffect, useId, useRef } from 'react'
import { X } from 'lucide-react'

// Shared dialog shell for the task form and the delete confirmation.
// Escape, the X button and a click on the dark backdrop all call onClose (same as Cancel).
// Put data-autofocus on the element that should receive focus when the dialog opens.
export default function Modal({ title, onClose, children }) {
  const titleId = useId()
  const dialogRef = useRef(null)
  const openerRef = useRef(document.activeElement)
  const onCloseRef = useRef(onClose)
  onCloseRef.current = onClose // always call the newest onClose without re-running the effect

  useEffect(() => {
    const opener = openerRef.current
    const onKeyDown = (event) => {
      if (event.key === 'Escape') onCloseRef.current()
    }
    document.addEventListener('keydown', onKeyDown)
    document.body.classList.add('modal-open')
    dialogRef.current.querySelector('[data-autofocus]')?.focus()

    return () => {
      document.removeEventListener('keydown', onKeyDown)
      document.body.classList.remove('modal-open')
      opener?.focus?.() // give focus back to the button that opened the dialog
    }
  }, [])

  return (
    <div
      className="modal-backdrop"
      onMouseDown={(event) => {
        if (event.target === event.currentTarget) onClose()
      }}
    >
      <div className="modal" role="dialog" aria-modal="true" aria-labelledby={titleId} ref={dialogRef}>
        <div className="modal-header">
          <h2 id={titleId}>{title}</h2>
          <button type="button" className="icon-button" aria-label="Close" onClick={onClose}>
            <X size={20} aria-hidden="true" />
          </button>
        </div>
        {children}
      </div>
    </div>
  )
}
```

src/components/Sidebar.jsx

```jsx
import { ChevronsLeft, ChevronsRight, CheckCheck, CircleCheck, Circle, Clock, LayoutDashboard, ListChecks, User, X } from 'lucide-react'

const NAV_ITEMS = [
  { label: 'Dashboard', icon: LayoutDashboard },
  { label: 'All Tasks', icon: ListChecks },
  { label: 'To Do', icon: Circle },
  { label: 'In Progress', icon: Clock },
  { label: 'Completed', icon: CircleCheck },
]

// Desktop: always visible, can shrink to icons. Mobile (under 768px): a drawer opened by the hamburger.
export default function Sidebar({ active, onNavigate, collapsed, onToggleCollapse, open, onClose }) {
  const className = ['sidebar', collapsed && 'collapsed', open && 'open'].filter(Boolean).join(' ')

  return (
    <aside className={className}>
      <div className="sidebar-header">
        <span className="logo" aria-hidden="true">
          <CheckCheck size={20} />
        </span>
        <span className="brand-name">TaskFlow</span>
        <button type="button" className="icon-button sidebar-close" aria-label="Close menu" onClick={onClose}>
          <X size={20} aria-hidden="true" />
        </button>
      </div>

      <nav className="sidebar-nav" aria-label="Main navigation">
        {NAV_ITEMS.map(({ label, icon: Icon }) => (
          <button
            key={label}
            type="button"
            className={label === active ? 'nav-item active' : 'nav-item'}
            aria-current={label === active ? 'page' : undefined}
            aria-label={label}
            title={label}
            onClick={() => onNavigate(label)}
          >
            <Icon size={20} aria-hidden="true" />
            <span className="nav-label">{label}</span>
          </button>
        ))}
      </nav>

      <div className="sidebar-footer">
        <div className="profile">
          <span className="avatar" aria-hidden="true">
            <User size={18} />
          </span>
          <div className="profile-text">
            <p className="profile-name">TaskFlow User</p>
            <p className="profile-note">Personal workspace</p>
          </div>
        </div>
        <button
          type="button"
          className="icon-button collapse-button"
          aria-label={collapsed ? 'Expand sidebar' : 'Collapse sidebar'}
          aria-expanded={!collapsed}
          title={collapsed ? 'Expand sidebar' : 'Collapse sidebar'}
          onClick={onToggleCollapse}
        >
          {collapsed ? <ChevronsRight size={20} aria-hidden="true" /> : <ChevronsLeft size={20} aria-hidden="true" />}
        </button>
      </div>
    </aside>
  )
}
```

src/components/StorageBanner.jsx

```jsx
import { TriangleAlert, X } from 'lucide-react'

export default function StorageBanner({ warning, onReset, onDismiss }) {
  return (
    <div className="banner" role="alert">
      <TriangleAlert size={20} aria-hidden="true" />
      <p>{warning.message}</p>
      {warning.canReset && (
        <button type="button" className="button button-secondary" onClick={onReset}>
          Reset storage
        </button>
      )}
      <button type="button" className="icon-button" aria-label="Dismiss warning" onClick={onDismiss}>
        <X size={18} aria-hidden="true" />
      </button>
    </div>
  )
}
```

src/components/SummaryCards.jsx

```jsx
import { CircleCheck, Clock, ListChecks, ListTodo } from 'lucide-react'

// Always counts ALL tasks, no matter which search or filters are active.
export default function SummaryCards({ tasks }) {
  const countOf = (status) => tasks.filter((task) => task.status === status).length
  const cards = [
    { label: 'Total Tasks', count: tasks.length, icon: ListChecks, tone: 'primary' },
    { label: 'To Do', count: countOf('To Do'), icon: ListTodo, tone: 'neutral' },
    { label: 'In Progress', count: countOf('In Progress'), icon: Clock, tone: 'warning' },
    { label: 'Completed', count: countOf('Completed'), icon: CircleCheck, tone: 'success' },
  ]

  return (
    <section className="summary" aria-label="Task summary">
      {cards.map(({ label, count, icon: Icon, tone }) => (
        <div key={label} className="summary-card">
          <span className={`summary-icon tone-${tone}`}>
            <Icon size={22} aria-hidden="true" />
          </span>
          <div>
            <p className="summary-label">{label}</p>
            <p className="summary-count">{count}</p>
          </div>
        </div>
      ))}
    </section>
  )
}
```

src/components/TaskCard.jsx

```jsx
import { Pencil, Trash2 } from 'lucide-react'
import { STATUSES } from '../utils/validation'
import { formatDueDate, formatTimestamp, isOverdue } from '../utils/dates'

export default function TaskCard({ task, today, onEdit, onDelete, onStatusChange }) {
  const overdue = isOverdue(task, today)
  const dueToday = task.dueDate === today && task.status !== 'Completed'
  const className = ['task-card', overdue && 'overdue', task.status === 'Completed' && 'completed']
    .filter(Boolean)
    .join(' ')

  return (
    <li className={className}>
      <div className="task-main">
        <div className="task-head">
          <h3 className="task-title">{task.title}</h3>
          <span className={`badge priority-${task.priority.toLowerCase()}`} title={`Priority: ${task.priority}`}>
            {task.priority}
          </span>
          <span className={`badge status-${task.status.toLowerCase().replace(' ', '-')}`} title={`Status: ${task.status}`}>
            {task.status}
          </span>
          {overdue && <span className="badge badge-overdue">Overdue</span>}
          {dueToday && <span className="badge badge-due-today">Due today</span>}
        </div>

        {task.description && <p className="task-description">{task.description}</p>}

        <p className="task-meta">
          {task.dueDate && (
            <span className={overdue ? 'due-date overdue-text' : 'due-date'}>Due: {formatDueDate(task.dueDate)}</span>
          )}
          <span>Created: {formatTimestamp(task.createdAt)}</span>
        </p>
      </div>

      <div className="task-actions">
        <select
          className="status-select"
          aria-label={`Change status of ${task.title}`}
          value={task.status}
          onChange={(event) => onStatusChange(task.id, event.target.value)}
        >
          {STATUSES.map((status) => (
            <option key={status} value={status}>
              {status}
            </option>
          ))}
        </select>
        <button type="button" className="button button-secondary" aria-label={`Edit ${task.title}`} onClick={() => onEdit(task.id)}>
          <Pencil size={16} aria-hidden="true" />
          Edit
        </button>
        <button type="button" className="button button-danger-outline" aria-label={`Delete ${task.title}`} onClick={() => onDelete(task.id)}>
          <Trash2 size={16} aria-hidden="true" />
          Delete
        </button>
      </div>
    </li>
  )
}
```

src/components/TaskFormModal.jsx

```jsx
import { useState } from 'react'
import Modal from './Modal'
import {
  DESCRIPTION_MAX,
  PRIORITIES,
  STATUSES,
  TITLE_MAX,
  countChars,
  validateTask,
} from '../utils/validation'

const EMPTY_FORM = { title: '', description: '', priority: 'Medium', status: 'To Do', dueDate: '' }

// One form for both "New Task" (no task prop) and "Edit Task" (task prop).
// The modal is only mounted while open, so the form starts fresh every time it opens.
export default function TaskFormModal({ task, onSave, onClose }) {
  const isEdit = Boolean(task)
  const [values, setValues] = useState(
    task
      ? {
          title: task.title,
          description: task.description,
          priority: task.priority,
          status: task.status,
          dueDate: task.dueDate || '',
        }
      : EMPTY_FORM,
  )
  const [submitted, setSubmitted] = useState(false)

  // Errors appear on the first submit, then stay live while the user edits.
  const errors = submitted ? validateTask(values) : {}

  const titleLength = countChars(values.title.trim())
  const descriptionLength = countChars(values.description.trim())

  function handleChange(event) {
    setValues({ ...values, [event.target.name]: event.target.value })
  }

  function handleSubmit(event) {
    event.preventDefault()
    setSubmitted(true)
    const found = validateTask(values)
    const firstInvalid = Object.keys(found)[0]
    if (firstInvalid) {
      event.currentTarget.elements.namedItem(firstInvalid)?.focus()
      return // invalid: do not save, keep the modal open
    }
    onSave(values)
  }

  const invalid = (field) => (errors[field] ? 'true' : 'false')

  return (
    <Modal title={isEdit ? 'Edit Task' : 'New Task'} onClose={onClose}>
      <form onSubmit={handleSubmit} noValidate>
        <div className="modal-body">
          <div className="field">
            <label htmlFor="task-title">Task Title</label>
            <input
              id="task-title"
              name="title"
              type="text"
              placeholder="e.g. Write Weekly Report"
              value={values.title}
              onChange={handleChange}
              aria-required="true"
              aria-invalid={invalid('title')}
              aria-describedby="task-title-count task-title-error"
              autoComplete="off"
```

```
72              data-autofocus
73            />
74            <div className="field-meta">
75              <span>Required, 3 to 80 characters</span>
76              <span id="task-title-count" className={titleLength > TITLE_MAX ? 'counter counter-over' :
   'counter'}>
77                {titleLength} / {TITLE_MAX}
78              </span>
79            </div>
80            {errors.title && (
81              <p id="task-title-error" className="field-error">
82                {errors.title}
83              </p>
84            )}
85          </div>
86
87          <div className="field">
88            <label htmlFor="task-description">Description</label>
89            <textarea
90              id="task-description"
91              name="description"
92              rows={4}
93              placeholder="Optional details about the task"
94              value={values.description}
95              onChange={handleChange}
96              aria-invalid={invalid('description')}
97              aria-describedby="task-description-count task-description-error"
98            />
99            <div className="field-meta">
100             <span>Optional</span>
101             <span
102               id="task-description-count"
103               className={descriptionLength > DESCRIPTION_MAX ? 'counter counter-over' : 'counter'}
104             >
105               {descriptionLength} / {DESCRIPTION_MAX}
106             </span>
107           </div>
108           {errors.description && (
109             <p id="task-description-error" className="field-error">
110               {errors.description}
111             </p>
112           )}
113         </div>
114
115         <div className="field-row">
116           <div className="field">
117             <label htmlFor="task-priority">Priority</label>
118             <select
119               id="task-priority"
120               name="priority"
121               value={values.priority}
122               onChange={handleChange}
123               aria-invalid={invalid('priority')}
124               aria-describedby="task-priority-error"
125             >
126               {PRIORITIES.map((priority) => (
127                 <option key={priority} value={priority}>
128                   {priority}
129                 </option>
130               ))}
131             </select>
132             {errors.priority && (
133               <p id="task-priority-error" className="field-error">
134                 {errors.priority}
135               </p>
136             )}
137           </div>
138
139           <div className="field">
140             <label htmlFor="task-status">Status</label>
141             <select
142               id="task-status"
143               name="status"
144               value={values.status}
```

```
            onChange={handleChange}
            aria-invalid={invalid('status')}
            aria-describedby="task-status-error"
          >
            {STATUSES.map((status) => (
              <option key={status} value={status}>
                {status}
              </option>
            ))}
          </select>
          {errors.status && (
            <p id="task-status-error" className="field-error">
              {errors.status}
            </p>
          )}
        </div>
      </div>

      <div className="field">
        <label htmlFor="task-dueDate">Due Date</label>
        <input
          id="task-dueDate"
          name="dueDate"
          type="date"
          value={values.dueDate}
          onChange={handleChange}
          aria-invalid={invalid('dueDate')}
          aria-describedby="task-dueDate-hint task-dueDate-error"
        />
        <div className="field-meta">
          <span id="task-dueDate-hint">Optional</span>
        </div>
        {errors.dueDate && (
          <p id="task-dueDate-error" className="field-error">
            {errors.dueDate}
          </p>
        )}
      </div>
    </div>

    <div className="modal-footer">
      <button type="button" className="button button-secondary" onClick={onClose}>
        Cancel
      </button>
      <button type="submit" className="button button-primary">
        {isEdit ? 'Save Changes' : 'Create Task'}
      </button>
    </div>
  </form>
 </Modal>
 )
}
```

src/components/TaskList.jsx

```jsx
import { ClipboardList, FilterX, SearchX } from 'lucide-react'
import EmptyState from './EmptyState'
import TaskCard from './TaskCard'

// Shows the task cards, or the right empty state when there is nothing to show.
export default function TaskList({
  totalCount,
  tasks,
  search,
  today,
  onEdit,
  onDelete,
  onStatusChange,
  onCreate,
  onClearSearch,
  onClearFilters,
}) {
  const query = search.trim()

  if (totalCount === 0) {
    return (
      <EmptyState
        icon={ClipboardList}
        title="No tasks yet"
        text="Tasks you create will appear here."
        actionLabel="Create your first task"
        onAction={onCreate}
      />
    )
  }

  if (tasks.length === 0 && query !== '') {
    return (
      <EmptyState
        icon={SearchX}
        title={`No tasks match "${query}"`}
        text="Try a different search term."
        actionLabel="Clear search"
        onAction={onClearSearch}
      />
    )
  }

  if (tasks.length === 0) {
    return (
      <EmptyState
        icon={FilterX}
        title="No tasks match the current filters"
        text="Try changing or clearing your filters."
        actionLabel="Clear filters"
        onAction={onClearFilters}
      />
    )
  }

  return (
    <>
      <p className="result-count">
        Showing {tasks.length} of {totalCount} {totalCount === 1 ? 'task' : 'tasks'}
      </p>
      <ul className="task-list">
        {tasks.map((task) => (
          <TaskCard key={task.id} task={task} today={today} onEdit={onEdit} onDelete={onDelete}
 onStatusChange={onStatusChange} />
        ))}
      </ul>
    </>
  )
}
```

src/components/TaskToolbar.jsx

```jsx
import { Search } from 'lucide-react'
import { PRIORITIES, STATUSES } from '../utils/validation'
import { SORT_OPTIONS } from '../utils/tasks'

// The search box here and the one in the top bar share the same state (search in App).
export default function TaskToolbar({
  search,
  onSearchChange,
  status,
  onStatusChange,
  priority,
  onPriorityChange,
  sort,
  onSortChange,
  onClear,
}) {
  return (
    <section className="toolbar" aria-label="Task filters">
      <div className="field toolbar-search">
        <label htmlFor="toolbar-search">Search</label>
        <div className="search-box">
          <Search size={18} aria-hidden="true" />
          <input
            id="toolbar-search"
            type="search"
            placeholder="Search by title or description"
            value={search}
            onChange={(event) => onSearchChange(event.target.value)}
          />
        </div>
      </div>

      <div className="field">
        <label htmlFor="filter-status">Status</label>
        <select id="filter-status" value={status} onChange={(event) =>
  onStatusChange(event.target.value)}>
          <option value="All">All Statuses</option>
          {STATUSES.map((value) => (
            <option key={value} value={value}>
              {value}
            </option>
          ))}
        </select>
      </div>

      <div className="field">
        <label htmlFor="filter-priority">Priority</label>
        <select id="filter-priority" value={priority} onChange={(event) =>
  onPriorityChange(event.target.value)}>
          <option value="All">All Priorities</option>
          {PRIORITIES.map((value) => (
            <option key={value} value={value}>
              {value}
            </option>
          ))}
        </select>
      </div>

      <div className="field">
        <label htmlFor="sort-by">Sort by</label>
        <select id="sort-by" value={sort} onChange={(event) => onSortChange(event.target.value)}>
          {SORT_OPTIONS.map((option) => (
            <option key={option.value} value={option.value}>
              {option.label}
            </option>
          ))}
        </select>
      </div>

      <button type="button" className="button button-secondary" onClick={onClear}>
        Clear Filters
```

```
70        </button>
71      </section>
72    )
73  }
```

src/components/Toast.jsx

```jsx
import { useEffect } from 'react'
import { CircleAlert, CircleCheck, X } from 'lucide-react'

// Pass a new key from the parent for every toast so the 3-second timer restarts.
export default function Toast({ message, type, onClose }) {
  useEffect(() => {
    const timer = setTimeout(onClose, 3000)
    return () => clearTimeout(timer)
  }, [onClose])

  const Icon = type === 'error' ? CircleAlert : CircleCheck
  return (
    <div className={`toast toast-${type}`} role={type === 'error' ? 'alert' : 'status'}>
      <Icon size={20} aria-hidden="true" />
      <span className="toast-message">{message}</span>
      <button type="button" className="icon-button toast-close" aria-label="Dismiss notification" onClick={onClose}>
        <X size={18} aria-hidden="true" />
      </button>
    </div>
  )
}
```

src/components/TopBar.jsx

```jsx
import { Menu, Search } from 'lucide-react'
import { formatToday } from '../utils/dates'

export default function TopBar({ title, search, onSearchChange, onNewTask, onOpenMenu }) {
  return (
    <header className="topbar">
      <button type="button" className="icon-button menu-button" aria-label="Open menu" onClick={onOpenMenu}>
        <Menu size={22} aria-hidden="true" />
      </button>
      <div className="topbar-title">
        <h1>{title}</h1>
        <p className="current-date">{formatToday()}</p>
      </div>
      <div className="topbar-search">
        <Search size={18} aria-hidden="true" />
        <input
          id="topbar-search"
          type="search"
          placeholder="Search tasks..."
          aria-label="Search tasks"
          value={search}
          onChange={(event) => onSearchChange(event.target.value)}
        />
      </div>
      <button type="button" className="button button-primary" onClick={onNewTask}>
        + New Task
      </button>
    </header>
  )
}
```

src/index.css

```
1  /* Base styles: colour palette, reset, buttons and form controls shared by every component. */
2
3  :root {
4    --bg: #f5f7fb;
5    --surface: #ffffff;
6    --text: #1f2937;
7    --text-muted: #6b7280;
8    --primary: #4f46e5;
9    --primary-dark: #4338ca;
10    --success: #16a34a;
11    --warning: #d97706;
12    --error: #dc2626;
13    --border: #e5e7eb;
14    --border-strong: #d1d5db;
15
16    /* Soft tints and darker text colours for badges (keeps text readable) */
17    --primary-tint: #e0e7ff;
18    --primary-ink: #3730a3;
19    --success-tint: #dcfce7;
20    --success-ink: #166534;
21    --warning-tint: #fef3c7;
22    --warning-ink: #92400e;
23    --error-tint: #fee2e2;
24    --error-ink: #991b1b;
25    --neutral-tint: #f3f4f6;
26    --neutral-ink: #374151;
27
28    --radius: 12px;
29    --shadow-sm: 0 1px 2px rgba(16, 24, 40, 0.06), 0 1px 3px rgba(16, 24, 40, 0.08);
30    --shadow-lg: 0 20px 40px rgba(16, 24, 40, 0.2);
31    --control-height: 40px;
32
33    font-family: -apple-system, BlinkMacSystemFont, 'Segoe UI', Roboto, 'Helvetica Neue', Arial, sans-
    serif;
34    font-size: 16px;
35    line-height: 1.5;
36    color: var(--text);
37  }
38
39  *,
40  *::before,
41  *::after {
42    box-sizing: border-box;
43  }
44
45  body {
46    margin: 0;
47    background: var(--bg);
48    color: var(--text);
49  }
50
51  body.modal-open {
52    overflow: hidden;
53  }
54
55  h1,
56  h2,
57  h3,
58  p {
59    margin: 0;
60  }
61
62  button,
63  input,
64  select,
65  textarea {
66    font: inherit;
67    color: inherit;
68  }
69
70  :focus-visible {
```

```
  outline: 3px solid rgba(79, 70, 229, 0.45);
  outline-offset: 2px;
}

/* ---------- Buttons ---------- */

.button {
  display: inline-flex;
  align-items: center;
  justify-content: center;
  gap: 6px;
  min-height: var(--control-height);
  padding: 0 16px;
  border: 1px solid transparent;
  border-radius: 8px;
  font-size: 14px;
  font-weight: 600;
  white-space: nowrap;
  cursor: pointer;
  transition: background-color 0.15s, border-color 0.15s;
}

.button-primary {
  background: var(--primary);
  color: #fff;
}
.button-primary:hover {
  background: var(--primary-dark);
}

.button-secondary {
  background: var(--surface);
  border-color: var(--border-strong);
  color: var(--text);
}
.button-secondary:hover {
  background: var(--neutral-tint);
}

.button-danger {
  background: var(--error);
  color: #fff;
}
.button-danger:hover {
  background: #b91c1c;
}

.button-danger-outline {
  background: var(--surface);
  border-color: #fca5a5;
  color: var(--error);
}
.button-danger-outline:hover {
  background: var(--error-tint);
}

.icon-button {
  display: inline-flex;
  align-items: center;
  justify-content: center;
  flex-shrink: 0;
  width: var(--control-height);
  height: var(--control-height);
  padding: 0;
  border: 0;
  border-radius: 8px;
  background: transparent;
  color: var(--text-muted);
  cursor: pointer;
}
.icon-button:hover {
  background: var(--neutral-tint);
  color: var(--text);
}
```

```
145
146 /* ---------- Form controls ---------- */
147
148 .field {
149   display: flex;
150   flex-direction: column;
151   gap: 6px;
152   min-width: 0;
153 }
154
155 .field > label {
156   font-size: 14px;
157   font-weight: 600;
158 }
159
160 input[type='text'],
161 input[type='search'],
162 input[type='date'],
163 select,
164 textarea {
165   width: 100%;
166   min-width: 0;
167   min-height: var(--control-height);
168   padding: 8px 12px;
169   border: 1px solid var(--border-strong);
170   border-radius: 8px;
171   background: var(--surface);
172   font-size: 15px;
173 }
174
175 textarea {
176   resize: vertical;
177 }
178
179 input[aria-invalid='true'],
180 select[aria-invalid='true'],
181 textarea[aria-invalid='true'] {
182   border-color: var(--error);
183 }
184
185 .field-meta {
186   display: flex;
187   justify-content: space-between;
188   gap: 12px;
189   font-size: 12px;
190   color: var(--text-muted);
191 }
192
193 .counter {
194   font-variant-numeric: tabular-nums;
195 }
196 .counter-over {
197   color: var(--error);
198   font-weight: 700;
199 }
200
201 .field-error {
202   font-size: 13px;
203   font-weight: 500;
204   color: var(--error);
205 }
206
207 /* ---------- Badges ---------- */
208
209 .badge {
210   display: inline-flex;
211   align-items: center;
212   padding: 1px 10px;
213   border-radius: 999px;
214   font-size: 12px;
215   font-weight: 600;
216   line-height: 20px;
217   white-space: nowrap;
218 }
```

```
219
220  .priority-low,
221  .status-completed {
222    background: var(--success-tint);
223    color: var(--success-ink);
224  }
225  .priority-medium,
226  .badge-due-today {
227    background: var(--warning-tint);
228    color: var(--warning-ink);
229  }
230  .priority-high,
231  .badge-overdue {
232    background: var(--error-tint);
233    color: var(--error-ink);
234  }
235  .status-to-do {
236    background: var(--neutral-tint);
237    color: var(--neutral-ink);
238  }
239  .status-in-progress {
240    background: var(--primary-tint);
241    color: var(--primary-ink);
242  }
243  .badge-overdue {
244    background: var(--error);
245    color: #fff;
246  }
247
248  @media (max-width: 768px) {
249    :root {
250      --control-height: 44px;
251    }
252  }
```

src/main.jsx

```jsx
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

src/utils/dates.js

```
1  // Date helpers. A due date is a plain "YYYY-MM-DD" string that always means a LOCAL calendar day.
2  // We never call new Date("2026-10-26"): that parses as UTC and can shift the day.
3
4  const DAYS_IN_MONTH = [31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31]
5  const isLeapYear = (year) => (year % 4 === 0 && year % 100 !== 0) || year % 400 === 0
6  const pad = (number) => String(number).padStart(2, '0')
7
8  // True only for real calendar dates written exactly as YYYY-MM-DD.
9  // 2026-02-29 and 2026-13-01 are invalid, 2028-02-29 is valid.
10 export function isValidDateString(value) {
11   if (typeof value !== 'string') return false
12   const match = /^(\d{4})-(\d{2})-(\d{2})$/.exec(value)
13   if (!match) return false
14   const year = Number(match[1])
15   const month = Number(match[2])
16   const day = Number(match[3])
17   if (month < 1 || month > 12 || day < 1) return false
18   const lastDay = month === 2 && isLeapYear(year) ? 29 : DAYS_IN_MONTH[month - 1]
19   return day <= lastDay
20 }
21
22 // Today's local date as YYYY-MM-DD. The "now" argument exists so tests can pass a fixed date.
23 export function todayString(now = new Date()) {
24   return `${now.getFullYear()}-${pad(now.getMonth() + 1)}-${pad(now.getDate())}`
25 }
26
27 // TF-REQ-006: overdue = due date before today AND not Completed. Due today is NOT overdue.
28 // YYYY-MM-DD strings compare correctly with < because they are zero-padded.
29 export function isOverdue(task, today = todayString()) {
30   return Boolean(task.dueDate) && task.dueDate < today && task.status !== 'Completed'
31 }
32
33 const SHORT_DATE = { month: 'short', day: 'numeric', year: 'numeric' }
34
35 // "2026-10-26" -> "Oct 26, 2026"
36 export function formatDueDate(dateString) {
37   const [year, month, day] = dateString.split('-').map(Number)
38   const date = new Date(2000, 0, 1)
39   date.setFullYear(year, month - 1, day) // setFullYear keeps years 0-99 as written
40   return date.toLocaleDateString('en-US', SHORT_DATE)
41 }
42
43 // ISO timestamp -> "Oct 26, 2026" (in the browser's local time zone)
44 export function formatTimestamp(isoString) {
45   return new Date(isoString).toLocaleDateString('en-US', SHORT_DATE)
46 }
47
48 // "Monday, October 26, 2026" for the top bar
49 export function formatToday(now = new Date()) {
50   return now.toLocaleDateString('en-US', { weekday: 'long', year: 'numeric', month: 'long', day:
   'numeric' })
51 }
```

src/utils/storage.js

```
import { normalizeValues } from './tasks.js'
import { validateTask } from './validation.js'

export const STORAGE_KEY = 'taskflow_tasks'
export const BACKUP_KEY = 'taskflow_tasks_backup'

const CORRUPTED = 'Saved task data was corrupted and could not be loaded. A backup was kept.'

function keepBackup(raw) {
  try {
    localStorage.setItem(BACKUP_KEY, raw)
  } catch {
    // Nothing more we can do if storage is full or blocked.
  }
}

// A saved entry is kept only if it passes the same rules as the form, has a string id and a real createdAt.
function toSavedTask(item, seenIds) {
  if (!item || typeof item !== 'object') return null
  if (typeof item.id !== 'string' || item.id === '' || seenIds.has(item.id)) return null
  if (Object.keys(validateTask(item)).length > 0) return null
  if (!Number.isFinite(Date.parse(item.createdAt))) return null
  const updatedAt = Number.isFinite(Date.parse(item.updatedAt)) ? item.updatedAt : item.createdAt
  return { id: item.id, ...normalizeValues(item), createdAt: item.createdAt, updatedAt }
}

// Never throws. Returns { tasks, warning } where warning is null or { message, canReset }.
export function loadTasks() {
  let raw
  try {
    raw = localStorage.getItem(STORAGE_KEY)
  } catch {
    return {
      tasks: [],
      warning: { message: 'Browser storage is unavailable, so tasks cannot be loaded or saved.', canReset: false },
    }
  }
  if (raw === null) return { tasks: [], warning: null }

  let data
  try {
    data = JSON.parse(raw)
  } catch {
    data = null
  }
  if (!Array.isArray(data)) {
    keepBackup(raw)
    return { tasks: [], warning: { message: CORRUPTED, canReset: true } }
  }

  const seenIds = new Set()
  const tasks = []
  for (const item of data) {
    const task = toSavedTask(item, seenIds)
    if (task) {
      seenIds.add(task.id)
      tasks.push(task)
    }
  }

  const skipped = data.length - tasks.length
  if (skipped === 0) return { tasks, warning: null }
  keepBackup(raw)
  const noun = skipped === 1 ? 'entry was' : 'entries were'
  return {
    tasks,
    warning: { message: `${skipped} saved ${noun} invalid and skipped. A backup was kept.`, canReset: false },
  }
```

```
69 }
70
71 // Returns true if the write worked, false if the browser refused it.
72 export function saveTasks(tasks) {
73   try {
74     localStorage.setItem(STORAGE_KEY, JSON.stringify(tasks))
75     return true
76   } catch {
77     return false
78   }
79 }
80
81 // Used by the "Reset storage" button. The backup copy is kept on purpose.
82 export function resetStorage() {
83   try {
84     localStorage.removeItem(STORAGE_KEY)
85   } catch {
86     // Ignore: storage is unavailable.
87   }
88 }
```

src/utils/tasks.js

```
1  // Helpers for building, searching, filtering and sorting tasks. No React in here.
2
3  export const SORT_OPTIONS = [
4    { value: 'newest', label: 'Newest first' },
5    { value: 'oldest', label: 'Oldest first' },
6    { value: 'due-soonest', label: 'Due date: soonest' },
7    { value: 'due-latest', label: 'Due date: latest' },
8  ]
9
10 // crypto.randomUUID only exists on https:// and localhost, so keep a fallback.
11 export function newId() {
12   const uuid = globalThis.crypto?.randomUUID?.()
13   return uuid ?? `${Date.now()}-${Math.random().toString(36).slice(2, 10)}`
14 }
15
16 // The form values that get saved: title and description are trimmed, no date becomes "".
17 export function normalizeValues(values) {
18   return {
19     title: values.title.trim(),
20     description: (values.description ?? '').trim(),
21     priority: values.priority,
22     status: values.status,
23     dueDate: values.dueDate || '',
24   }
25 }
26
27 export function buildTask(values) {
28   const now = new Date().toISOString()
29   return { id: newId(), ...normalizeValues(values), createdAt: now, updatedAt: now }
30 }
31
32 // Search + status + priority combine with AND (TF-REQ-019), then the list is sorted.
33 export function filterAndSortTasks(tasks, { search, status, priority, sort }) {
34   const query = search.trim().toLowerCase()
35   const matches = tasks.filter(
36     (task) =>
37       (status === 'All' || task.status === status) &&
38       (priority === 'All' || task.priority === priority) &&
39       (query === '' || task.title.toLowerCase().includes(query) ||
   task.description.toLowerCase().includes(query)),
40   )
41
42   const created = (task) => Date.parse(task.createdAt)
43   const newestFirst = (a, b) => created(b) - created(a)
44   // direction 1 = soonest first, -1 = latest first. Tasks without a due date always go last.
45   const byDueDate = (direction) => (a, b) => {
46     if (!a.dueDate && !b.dueDate) return newestFirst(a, b)
47     if (!a.dueDate) return 1
48     if (!b.dueDate) return -1
49     if (a.dueDate === b.dueDate) return newestFirst(a, b)
50     return (a.dueDate < b.dueDate ? -1 : 1) * direction
51   }
52   const comparators = {
53     newest: newestFirst,
54     oldest: (a, b) => created(a) - created(b),
55     'due-soonest': byDueDate(1),
56     'due-latest': byDueDate(-1),
57   }
58   return matches.sort(comparators[sort] ?? newestFirst)
59 }
```

src/utils/validation.js

```
import { isValidDateString } from './dates.js'

// The only values the app accepts (TF-REQ-004 and TF-REQ-005).
export const PRIORITIES = ['Low', 'Medium', 'High']
export const STATUSES = ['To Do', 'In Progress', 'Completed']

export const TITLE_MIN = 3
export const TITLE_MAX = 80
export const DESCRIPTION_MAX = 500

// Length in Unicode code points, so an emoji counts as 1 character (string.length would say 2).
export const countChars = (text) => [...text].length

// Checks a task form (or a saved task) and returns { fieldName: message }.
// An empty object means the values are valid. Used by create, edit and the storage loader.
export function validateTask(values) {
  const errors = {}

  // TF-REQ-001 / TF-REQ-002: trim first, then count characters.
  const titleLength = countChars(typeof values.title === 'string' ? values.title.trim() : '')
  if (titleLength === 0) errors.title = 'Title is required.'
  else if (titleLength < TITLE_MIN) errors.title = `Title must be at least ${TITLE_MIN} characters.`
  else if (titleLength > TITLE_MAX) errors.title = `Title must be ${TITLE_MAX} characters or fewer.`

  // TF-REQ-003: optional, up to 500 characters (leading/trailing spaces are not counted or saved).
  const description = values.description ?? ''
  if (typeof description !== 'string' || countChars(description.trim()) > DESCRIPTION_MAX) {
    errors.description = `Description must be ${DESCRIPTION_MAX} characters or fewer.`
  }

  // TF-REQ-004 / TF-REQ-005: only the listed values are allowed.
  if (!PRIORITIES.includes(values.priority)) errors.priority = 'Please select a valid priority.'
  if (!STATUSES.includes(values.status)) errors.status = 'Please select a valid status.'

  // TF-REQ-006: optional; if present it must be a real YYYY-MM-DD date. Past dates are fine.
  const { dueDate } = values
  if (dueDate !== undefined && dueDate !== null && dueDate !== '' && !isValidDateString(dueDate)) {
    errors.dueDate = 'Please enter a valid date (YYYY-MM-DD).'
  }

  return errors
}
```